

LAB RECORD

COMPILER DESIGN

19CSE401



Amrita School of Computing

Amrita Vishwa Vidyapeetham

Chennai – 601 103, Tamil Nadu, India.

BONAFIDE CERTIFICATE

University Reg. No : _____CH.EN.U4CSE22043_____

This is to certify that this is a bonafide record work done by

Miss._____Palavajjhala Neha Madhu

Manasa_____studying

B.Tech_____4th Year_____

INDEX

Experiment Number	Topic Name	Date	Page no
01	Implementation of Lexical Analyzer Using Lex Tool	7-07-2025	4
02	Program to eliminate left recursion and factoring from the given grammar	21-07-2025	9
03	Implementation of LL(1) parsing	11-09-2025	12
04	Parser Generation using YACC	25-09-2025	14
05	Implementation of Symbol Table	1-09-2025	15
06	Implementation of Intermediate Code Generation	8-09-2025	17
07	Implementation of Code Optimization Techniques	15-09-2025	19
08	Implementation of Target code generation	22-09-2025	21

Exercise 1

1.

Aim: LEX program to count the number of vowels and consonants in a given string

Code:

```
%{
    int vow_count=0;
    int const_count =0;
}%

%%
[aeiouAEIOU] {vow_count++;}
[a-zA-Z] {const_count++;}
%%
int yywrap(){}
int main()
{
    printf("Enter the string of vowels and consonants:");
    yylex();
    printf("Number of vowels are: %d\n", vow_count);
    printf("Number of consonants are: %d\n", const_count);
    return 0;
}
```

Output :

```
asecomputerlab@lab:~$ lex Pgm1.l
asecomputerlab@lab:~$ gcc lex.yy.c -lfl -o pgm1
asecomputerlab@lab:~$ ./pgm1
Enter the string of vowels and consonants:Madhu Manasa CSEA 22043
22043
Number of vowels are: 7
Number of consonants are: 8
asecomputerlab@lab:~$ |
```

2.

Aim: Lex code for password validation

Code:

```
1 /*lex program to count number of words*/
2 %{
3 #include<stdio.h>
4 #include<string.h>
5 int i = 0;
6 %}
7
8 /* Rules Section*/
9 %%
10 ([a-zA-Z0-9])*    {i++;} /* Rule for counting
11                          number of words*/
12
13 "\n" {printf("%d\n", i); i = 0;}
14 %%
15
16 int yywrap(void){}
17
18 int main()
19 {
20     // The function that starts the analysis
21     yylex();
22
23     return 0;
24 }
```

Output :

```
asecomputerlab@lab:~$ lex Pgm2.l
asecomputerlab@lab:~$ gcc lex.yy.c -lfl -o pgm2
asecomputerlab@lab:~$ ./pgm2
AseComputer@123

VALID
asecomputerlab@lab:~$ |
```

3.

Aim: Lex program to count the number of lines, spaces and tabs

Code :

```
%{
#include<stdio.h>
int lc=0,sc=0,tc=0,ch=0,wc=0;
}%

%%
[\\n] { lc++; ch+=yyleng;}
[ \\t] { sc++; ch+=yyleng;}
[^\\t] { tc++; ch+=yyleng;}
[^\\t\\n ]+ { wc++; ch+=yyleng;}
%%

int yywrap(){ return 1; }
|
int main(){
printf("Enter the Sentence : ");
yylex();
printf("Number of lines : %d\\n",lc);
printf("Number of spaces : %d\\n",sc);
printf("Number of tabs, words, charc : %d , %d , %d\\n",tc,wc,ch);

return 0;
}
```

Output :

```
asecomputerlab@lab:~$ lex Pgm3.l
asecomputerlab@lab:~$ gcc lex.yy.c -lfl -o pgm3
asecomputerlab@lab:~$ ./pgm3
Enter the Sentence : Neha Madhu Manasa CSEA 4th Year
Number of lines : 1
Number of spaces : 5
Number of tabs, words, charc : 0 , 6 , 32
asecomputerlab@lab:~$ |
```

4.

Aim: Lex Program to Identify and Count Positive and Negative Numbers

Code:

```
%{  
    int postiveno=0;  
    int negativeno=0;  
    int positivefractions=0;  
    int negativefractions=0;  
}%  
  
DIGIT [0-9]  
%%  
  
\+?[DIGIT]+          postiveno++;  
-[DIGIT]+           negativeno++;  
  
\+?[DIGIT]*\.[DIGIT]+ positivefractions++;  
-[DIGIT]*\.[DIGIT]+  negativefractions++;  
.  
;  
%%  
  
|  
int main()  
{  
    yylex();  
    printf("\nNo. of positive numbers: %d", postiveno);  
    printf("\nNo. of Negative numbers: %d", negativeno);  
    printf("\nNo. of Positive numbers in fractions: %d", positivefractions);  
    printf("\nNo. of Negative numbers in fractions: %d\n", negativefractions);  
    return 0;  
}
```

Output :

```
asecomputerlab@lab:~$ lex Pgm4.l  
asecomputerlab@lab:~$ gcc lex.yy.c -lfl -o pgm4  
asecomputerlab@lab:~$ ./pgm4  
1 2 3 4.5 -6 -5  
  
No. of positive numbers: 3  
No. of Negative numbers: 2  
No. of Positive numbers in fractions: 1  
No. of Negative numbers in fractions: 0
```

5.

Aim: Lex program to identify the identifier

Code:

```
%{
#include <stdio.h>
%}

%%
^[a-zA-Z_][a-zA-Z0-9_]+ printf("Valid Identifier");
^[^a - zA - Z_] printf("Invalid Identifier");
.;
%%

int main()
{
    yylex();
}
```

Output :

```
asecomputerlab@lab:~$ lex Pgm5.l
asecomputerlab@lab:~$ gcc lex.yy.c -lfl -o pgm5
asecomputerlab@lab:~$ ./pgm5
$wer
Invalid Identifierwer
abc1_
Valid Identifier
asecomputerlab@lab:~$ |
```


Exercise 2

1. Aim: To implement eliminate left recursion and left factoring from the given grammar using C program.

Algorithm:

Left Factoring:

- Start the processes by getting the grammar and assigning it to the appropriate variables
- Find the common terminal and non-terminal elements and assign them in a separate grammar
- Display the new and modified grammar.

Code:

```
ex2.c
1 #include<stdio.h>
2 #include<string.h>
3 int main()
4 {
5     char gram[20],part1[20],part2[20],modifiedGram[20],newGram[20],tempGram[20];
6     int i,j=0,k=0,l=0,pos;
7     printf("Enter Production : A->");
8     gets(gram);
9     for(i=0;gram[i]!='\0';i++,j++)
10         part1[j]=gram[i];
11     part1[j]='\0';
12     for(j=++i,i=0;gram[j]!='\0';j++,i++)
13         part2[i]=gram[j];
14     part2[i]='\0';
15     for(i=0;i<strlen(part1)||i<strlen(part2);i++)
16     {
17         if(part1[i]==part2[i])
18         {
19             modifiedGram[k]=part1[i];
20             k++;
21
22             pos=i+1;
23         }
24     }
25     for(i=pos,j=0;part1[i]!='\0';i++,j++){
26         newGram[j]=part1[i];
27     }
28     newGram[j++]='|';
29     for(i=pos;part2[i]!='\0';i++,j++){
30         newGram[j]=part2[i];
31     }
32     modifiedGram[k]='X';
33     modifiedGram[k++]='\0';
34     newGram[j]='\0';
35     printf("\n A->%s",modifiedGram);
36     printf("\n X->%s\n",newGram);
37 }
```

Output:

```
asecomputerlab@lab:~$ gcc Left_factoring_Ex2.c
Left_factoring_Ex2.c: In function 'main':
Left_factoring_Ex2.c:9:2: warning: implicit declaration of function 'gets'; did you mean 'fgets'? [-Wimplicit-function-declaration]
  gets(gram);
  ^~~~~
  fgets
/tmp/ccJEtMB0.o: In function 'main':
Left_factoring_Ex2.c:(.text+0x5a): warning: the 'gets' function is dangerous and should not be used.
asecomputerlab@lab:~$ ./a.out
Enter Production : A->aE+bcD|aE+eIT
A->aE+X
X->bcD|eIT
```

Left recursion:

2. Aim: To implement left recursion using C.

Algorithm:

- Start the processes by getting the grammar and assigning it to the appropriate variables.
- Check if the given grammar has left recursion.
- Identify the alpha and beta elements in the production.
- Print the output according to the formula to remove left recursion

Code:

```
1 #include<stdio.h>
2 #include<string.h>
3 #define SIZE 10
4 int main () {
5     char non_terminal;
6     char beta,alpha;
7     int num;
8     char production[10][SIZE];
9     int index=3; /* starting of the string following "->" */
10    printf("Enter Number of Production : ");
11    scanf("%d",&num);
12    printf("Enter the grammar as E->E-A :\n");
13    for(int i=0;i<num;i++){
14        scanf("%s",production[i]);
15    }
16    for(int i=0;i<num;i++){
17        printf("\nGRAMMAR : : : %s",production[i]);
18        non_terminal=production[i][0];
19        if(non_terminal==production[i][index]) {
20            alpha=production[i][index+1];
21            printf(" is left recursive.\n");
22            while(production[i][index]!=0 && production[i][index]!='|')
23                index++;
24            if(production[i][index]!=0) {
25                beta=production[i][index+1];
26                printf("Grammar without left recursion:\n");
27                printf("%c->%c%c\'",non_terminal,beta,non_terminal);
28                printf("\n%c\'->%c%c\'|E\n",non_terminal,alpha,non_terminal);
29            }
30        } else
31            printf(" can't be reduced\n");
32    }
33    else
34        printf(" is not left recursive.\n");
35    index=3;
36    }
37    }
38    }
39    }
```

Output:

```
asecomputerlab@lab:~$ gcc Left_recursion_Ex2.c
asecomputerlab@lab:~$ ./a.out
Enter Number of Production : 2
Enter the grammar as E->E-A :

E->EA|A
A->A|B

GRAMMAR : : : E->EA|A is left recursive.
Grammar without left recursion:
E->AE'
E'->AE'|E

GRAMMAR : : : A->A|B is left recursive.
Grammar without left recursion:
A->BA'
A'->|A'|E
```

Result: The program to implement left factoring and left recursion has been successfully executed.

Exercise 3

Aim: To implement LL(1) parsing using C program.

Algorithm:

- 1) Read the input string.
- 2) Using predictive parsing table parse the given input using stack.
- 3) If stack [i] matches with token input string pop the token else shift it repeat the process until it reaches to \$.

Code :

[illegible]

Output :

```
asecomputerlab@ase-computer-lab:~$ gedit ll.c
asecomputerlab@ase-computer-lab:~$ gcc ll.c
asecomputerlab@ase-computer-lab:~$ ./a.out
```

Enter the input string: i*i+i

Stack	Input
\$bt	i*i+i\$
\$bcf	i*i+i\$
\$bci	i*i+i\$
\$bcf*	*i+i\$
\$bci	i+i\$
\$b	+i\$
\$bt+	+i\$
\$bcf	i\$
\$bci	i\$
\$b	\$

Exercise 4

Aim: To write a program in YACC for parser generation.

Algorithm:

- 1) Get the input from the user and Parse it token by token.
- 2) First identify the valid inputs that can be given for a program.
- 3) Define the precedence and the associativity of various operators like +,-,/,* etc.
- 4) Write codes for saving the answer into memory and displaying the result on the screen.
- 5) Write codes for performing various arithmetic operations.
- 6) Display the possible Error message that can be associated with this calculation.
- 7) Display the output on the screen else display the error message on the screen

Code :

```
% { int yylex();
% } % { #define YYSTYPE double #include <ctype.h> #include <stdio.h>
% } %token NUMBER %left '+' '-' %left '*' '/' %right UMINUS % % lines :lines expr '\n' { printf("%g\n", $2);
% } | lines '\n' /* E * /;
expr :expr '+' expr { $$ = $1 + $3;
% } | expr '-' expr { $$ = $1 - $3;
% } | expr '/' expr { $$ = $1 / $3;
% } | '(' expr ')' { $$ = $2;
% } | '-' expr %prec UMINUS { $$ = - $2;
% } | NUMBER;
% % yylex() { int c;
while((c = getchar()) == ' ');
if((c == '.') || (isdigit(c))) { ungetc(c, stdin);
scanf("%lf", & yylval);
return NUMBER;
% } return c;
% } int main() { yyparse();
return 1;
% } int yyerror() { return 1;
% } int yywrap() { return 1;
% }
```

Output :

```
hadoop@hadoop-virtual-machine:~/Desktop/Compiler design lab$ yacc cal.y
hadoop@hadoop-virtual-machine:~/Desktop/Compiler design lab$ gcc -o cal y.tab.c
y.tab.c: In function 'yyparse':
y.tab.c:1292:7: warning: implicit declaration of function 'yyerror'; did you mean
'an 'yyerrok'? [-Wimplicit-function-declaration]
    yyerror (YY_("syntax error"));
    ~~~~~~
yyerrok
cal.y: At top level:
cal.y:30:1: warning: return type defaults to 'int' [-Wimplicit-int]
    yylex()
    ~~~~~~
hadoop@hadoop-virtual-machine:~/Desktop/Compiler design lab$ ./cal
20+51
71
65-96
-31
245+658
903
210+65
275
```

Result: Thus the program in YACC for parser generation has been executed successfully.

Exercise 5

Aim: To implement Symbol Table.

Algorithm:

1. Start the Program.
2. Get the input from the user with the terminating symbol '\$'.
3. Allocate memory for the variable by dynamic memory allocation function.
4. If the next character of the symbol is an operator then only the memory is allocated.
5. While reading, the input symbol is inserted into symbol table along with its memory address.
6. The steps are repeated till "\$" is reached.
7. To reach a variable, enter the variable to the searched and symbol table has been
8. Checked for corresponding variable, the variable along its address is displayed as result.
9. Stop the program

Code :

```
#include<stdio.h> #include<math.h> #include<string.h> #include<ctype.h> #include<stdlib.h>
void main(){
int x=0, n, i=0,j=0;
void *mypointer,*T4Tutorials_address[5];
char ch,T4Tutorials_Search,T4Tutorials_Array2[15],T4Tutorials_Array3[15],c; printf("Input the expression ending with $ sign:");
while((c=getchar())!='$')
{
T4Tutorials_Array2[i]=c; i++;
}
n=i-1;
printf("Given Expression:");
i=0;
while(i<=n)
{
printf("%c",T4Tutorials_Array2[i]); i++;
}
printf("\n Symbol Table display\n"); printf("Symbol \t addr \t type"); while(j<=n)
{
c=T4Tutorials_Array2[j]; if(isalpha(toascii(c)))
{
mypointer=malloc(c); T4Tutorials_address[x]=mypointer; T4Tutorials_Array3[x]=c;
printf("\n%c \t %d \t identifier\n",c,mypointer); x++;
j++;
}
else
{
ch=c;
if(ch=='+'||ch=='-'||ch=='*'||ch=='/')
{
mypointer=malloc(ch); T4Tutorials_address[x]=mypointer; T4Tutorials_Array3[x]=ch;
printf("\n %c \t %d \t operator\n",ch,mypointer); x++;
j++;
}
}
}
}
```

Output :

```
hadoop@hadoop-virtual-machine:~/Desktop/Compiler design lab$ ./a.out
Input the expression ending with $ sign:w=a+b*c$
Given Expression:w=a+b*c
Symbol Table display
Symbol  addr    type
w       1840614016  identifier
=       1840614144  operator
a       1840614224  identifier
+       1840614336  operator
b       1840614400  identifier
*       1840614512  operator
c       1840614576  identifier
hadoop@hadoop-virtual-machine:~/Desktop/Compiler design lab$
```

Result: Thus the program to implement symbol table has been executed successfully.

Exercise 6

Aim: To implementation of intermediate code generation.

Algorithm:

- 1) Take the parse tree tokens from the syntax analyser.
- 2) Generate intermediate code using temp variable
- 3) Assign the final temp variable to initial variable

Code :

```
#include<stdio.h> #include<stdlib.h> #include<string.h>
int i = 1,
j = 0,
no = 0,
tmpch = 90;
char str [100],
left [15],
right [15];
void findopr();
void explore();
void fleft(int);
void fright(int);
struct exp { int pos;
char op;
} k [15];
void main() {
printf("\t\tINTERMEDIATE CODE GENERATION\n\n");
printf("Enter the Expression :");
scanf("%s", str);
printf("The intermediate code:\n");
findopr();
explore();
} void findopr() { for( i = 0; str[i] != '\0'; i++)
if(str[i] == ':') { k[j].pos = i;
k[j++].op = ':';
} for(
i = 0;
str[i] != '\0';
i++)
if(str[i] == '/') { k[j].pos = i;
k[j++].op = '/';
} for(
i = 0;
str[i] != '\0';
i++)
if(str[i] == '*') { k[j].pos = i;
k[j++].op = '*';
} for(
i = 0;
str[i] != '\0';
i++)
if(str[i] == '+') { k[j].pos = i;
k[j++].op = '+';
} for(
i = 0;
str[i] != '\0';
i++)
```

```

i ++
} if(str [i] == '-') { k [j].pos = i;
k [j++].op = '-';
} } void explore() { i = 1;
while(k [i].op != '\0') { fleft(k [i].pos);
fright(k [i].pos);
str [k[i].pos] = tmpch --;
printf(
"\t%c := %s%c%s\t\t",
str [k[i].pos],
left,
k [i].op,
right
);
};
printf("\n");
i ++;
} fright(-1);
if(no == 0) { fleft(strlen(str));
printf("\t%s := %s", right, left);
exit(0);
} printf("\t%s := %c", right, str [k[--i].pos]);
} void fleft(int x) { int w = 0,
flag = 0;
x --;
while(
x != -1 && str [x] != '+' && str [x] != '*' && str [x] != '=' && str [x] != '\0' && str [x] != ':' && str [x] != '/' && str [x] != '-' && str [x] != '.' && str [x] != ' ' && str [x] != '\n') { if(str [x] != '$' && flag == 0) { left [w++] = str [x];
left [w] = '\0';
str [x] = '$';
flag = 1;
} x --;
} } void fright(int x) { int w = 0,
flag = 0;
x ++;
while(
x != -1 && str [x] != '+' && str [x] != '*' && str [x] != '\0' && str [x] != '=' && str [x] != ':' && str [x] != '/' && str [x] != '-' && str [x] != '.' && str [x] != ' ' && str [x] != '\n') { if(str [x] != '$' && flag == 0) { right [w++] = str [x];
right [w] = '\0';
str [x] = '$';
flag = 1;
} x ++;
} }
}

```

Output :

```

hadoop@hadoop-virtual-machine:~/Desktop/Compiler design lab$ gedit icg.c
hadoop@hadoop-virtual-machine:~/Desktop/Compiler design lab$ gcc icg.c
hadoop@hadoop-virtual-machine:~/Desktop/Compiler design lab$ ./a.out
INTERMEDIATE CODE GENERATION

Enter the Expression :w:=a+b*c
The intermediate code:
    Z := b*c
    Y := a+Z
    W := Y
hadoop@hadoop-virtual-machine:~/Desktop/Compiler design lab$

```

Result: Thus, the program to implement intermediate code generation has been executed successfully.

Exercise 7

Aim: To implementation of Code Optimization Techniques

Algorithm:

- 1) Start the program
- 2) Declare the variables and functions.
- 3) Enter the expression and state it in the variable a, b, c.
- 4) Calculate the variables b & c with 'temp' and store it in f1 and f2.
- 5) If(f1=null && f2=null) then expression could not be optimized.
- 6) Print the results.
- 7) Stop the program.

Code :

```
#include<stdio.h> #include<string.h>
struct op
{
char l; char r[20];
} op[10],pr[10];
void main()
{
int a,i,k,j,n,z=0,m,q; char *p,*l;
char temp,t; char *tem;
printf("Enter the Number of Values:"); scanf("%d",&n);
for(i=0;i<n;i++)
{
printf("left: "); scanf(" %c",&op[i].l); printf("right: "); scanf(" %s",&op[i].r);
}
printf("Intermediate Code\n") ; for(i=0;i<n;i++)
{
printf("%c=",op[i].l);
printf("%s\n",op[i].r);
}
for(i=0;i<n-1;i++)
{
temp=op[i].l; for(j=0;j<n;j++)
{
p=strchr(op[j].r,temp); if(p)
{
pr[z].l=op[i].l;
strcpy(pr[z].r,op[i].r);
z++;
} } }
pr[z].l=op[n-1].l;
strcpy(pr[z].r,op[n-1].r); z++;
printf("\nAfter Dead Code Elimination\n"); for(k=0;k<z;k++)
{
printf("%c\t=",pr[k].l);
printf("%s\n",pr[k].r);
}
for(m=0;m<z;m++)
{
tem=pr[m].r; for(j=m+1;j<z;j++)
{
p=strstr(tem,pr[j].r); if(p)
{
t=pr[j].l; pr[j].l=pr[m].l; for(i=0;i<z;i++)
{
l=strchr(pr[i].r,t) ; if(l)
{
a=l-pr[i].r;
printf("pos: %d\n",a);
}
```

```

pr[i].r[a]=pr[m].l;
}}}}
printf("Eliminate Common Expression\n"); for(i=0;i<z;i++)
{
printf("%c\t=",pr[i].l);
printf("%s\n",pr[i].r);
}
for(i=0;i<z;i++)
{
for(j=i+1;j<z;j++)
{
q=strcmp(pr[i].r,pr[j].r);
if((pr[i].l==pr[j].l)&&!q)
{
pr[i].l='\0';
} } }
printf("Optimized Code\n");

for(i=0;i<z;i++)
{
if(pr[i].l!='\0')
{
printf("%c=",pr[i].l);
printf("%s\n",pr[i].r);
}
}
}
}

```

Output :

```

hadoop@hadoop-virtual-machine:~/Desktop/Compiler design lab$ ./a.o
Enter the Number of Values:3
left: a
right: S
left: b
right: a+c
left: c
right: c*5
Intermediate Code
a=S
b=a+c
c=c*5

After Dead Code Elimination
a      =S
c      =c*5
Eliminate Common Expression
a      =S
c      =c*5
Optimized Code
a=S
c=c*5
hadoop@hadoop-virtual-machine:~/Desktop/Compiler design lab$

```

Result: Thus, the program to implement code optimization has been executed successfully.

Exercise 8

Aim: To write a program that implements the target code generation

Algorithm:

1. Read input string
2. Consider each input string and convert it to machine code instructions using switch case
3. Load the input variables into new variables as operands and display them using "load"
4. With the help of arithmetic operation, we will display arithmetic operations like add, sub, div, mul for the respective operations in switch case
5. Generate 3 address code for each input variable.
6. If '=' is seen as arithmetic operation, then store the result in a variable and display it with "store".
7. Repeat this for each line in the input string.
8. Display the output which gives a transformed input string of assembly language code.

Code :

```
#include<stdio.h> #include<stdlib.h> #include<string.h>
int label[20];
int no=0;
int main()
{
    FILE *fp1,*fp2;
    char fname[10],op[10],ch;
    char operand1[8],operand2[8],result[8]; int i=0,j=0;
    printf("\n Enter filename of the intermediate code");
    scanf("%s",&fname); fp1=fopen(fname,"r"); fp2=fopen("target.txt","w"); if(fp1==NULL || fp2==NULL)
    {
        printf("\n Error opening the file"); exit(0);
    }
    while(!feof(fp1))
    {
        fprintf(fp2,"\n");
        fscanf(fp1,"%s",op); i++;
        if(check_label(i))
            fprintf(fp2,"\nlabel%d",i); if(strcmp(op,"print")==0)
        {
            fscanf(fp1,"%s",result);
            fprintf(fp2,"\n\t OUT %s",result);
        }
        if(strcmp(op,"goto")==0)
        {
            fscanf(fp1,"%s %s",operand1,operand2); fprintf(fp2,"\n\t JMP %s,label#%s",operand1,operand2); label[no++]=atoi(operand2);
        }
        if(strcmp(op,"[]")==0)
        {
            fscanf(fp1,"%s %s %s",operand1,operand2,result); fprintf(fp2,"\n\t STORE %s[%s],%s",operand1,operand2,result);
        }
        if(strcmp(op,"uminus")==0)
        {
            fscanf(fp1,"%s %s",operand1,result); fprintf(fp2,"\n\t LOAD -%s,R1",operand1); fprintf(fp2,"\n\t STORE R1,%s",result);
        }
        switch(op[0])
        {
            case '*': fscanf(fp1,"%s %s %s",operand1,operand2,result);
                    fprintf(fp2,"\n\t tLOAD",operand1); fprintf(fp2,"\n\t LOAD%R1",operand2); fprintf(fp2,"\n\t MUL R1,R0"); fprintf(fp2,"\n\t STORE R0,%s",result); break;
            case '+': fscanf(fp1,"%s %s %s",operand1,operand2,result); fprintf(fp2,"\n\t LOAD %s,R0",operand1); fprintf(fp2,"\n\t LOAD %s,R1",operand2); fprintf(fp2,"\n\t ADD R1,R0"); fpr
            case '-': fscanf(fp1,"%s %s %s",operand1,operand2,result); fprintf(fp2,"\n\t LOAD %s,R0",operand1); fprintf(fp2,"\n\t tLOAD %s,R1",operand2); fprintf(fp2,"\n\t SUB R1,R0"); fpr
            case '/': fscanf(fp1,"%s %s %s",operand1,operand2,result); fprintf(fp2,"\n\t LOAD %s,R0",operand1); fprintf(fp2,"\n\t LOAD %s,R1",operand2); fprintf(fp2,"\n\t DIV R1,R0"); fpr

            case '%': fscanf(fp1,"%s %s %s",operand1,operand2,result); fprintf(fp2,"\n\t LOAD %s,R0",operand1); fprintf(fp2,"\n\t LOAD %s,R1",operand2); fprintf(fp2,"\n\t DIV R1,R0"); fpr

            case '=': fscanf(fp1,"%s %s",operand1,result);
                    fprintf(fp2,"\n\t STORE %s %s",operand1,result); break;
        }
    }
}
```

```

case '>': j++;
fscanf(fp1,"%s %s %s",operand1,operand2,result); fprintf(fp2,"\n \t LOAD %s,R0",operand1); fprintf(fp2,"\n\t JGT %s,label#%s",operand2,result); label[no++] = atoi(result);
break;
case '<': fscanf(fp1,"%s %s %s",operand1,operand2,result); fprintf(fp2,"\n \t LOAD %s,R0",operand1); fprintf(fp2,"\n\t JLT %s,label#%d",operand2,result); label[no++] = atoi(result);
break;
}
}
fclose(fp2); fclose(fp1);
fp2=fopen("target.txt","r"); if(fp2==NULL)
{
printf("Error opening the file\n"); exit(0);
}
do
{
ch=fgetc(fp2); printf("%c",ch);
}while(ch!=EOF); fclose(fp1); return 0;
}
int check_label(int k)
{
int i; for(i=0;i<no;i++)
{
if(k==label[i]) return 1;
}
return 0;
}

```

Output :

```

hadoop@hadoop-virtual-machine:~/Desktop/Compiler design lab$ ./a.out

Enter filename of the intermediate codeInput.txt

LOAD t2,R0
LOAD t2,R1
DIV R1,R0
STORE R0,4U

LOAD -t2,R1
STORE R1,t2

OUT t2

LOAD t3,R0
LOAD t4,R1
ADD R1,R0
STORE R0,print

hadoop@hadoop-virtual-machine:~/Desktop/Compiler design lab$

```

Result: Thus, the program to implement target code generation has been successfully executed.